

Sprint 16 Structured Notes: Dates, Time, Audio, Video, and Media APIs

0. What Sprint 16 Is About

Sprint 16 teaches you how JavaScript works with browser features related to:

1. dates
2. time
3. audio
4. video
5. media file formats
6. MIME types
7. codecs
8. media controls
9. media streams
10. simple browser permission behavior

In earlier sprints, you learned how to select page elements, change the DOM, listen for events, update accessibility state, and validate forms. Sprint 16 adds another browser skill: using built-in browser APIs for time and media.

Main goal:

Work with browser date/time and media features without external libraries.

Simple meaning:

Sprint 16 teaches JavaScript how to understand time and control media.

Example date code:

```
const now = new Date();  
console.log(now.getFullYear());
```

Example audio code:

```
const audio = new Audio("sound.mp3");  
audio.play();
```

By the end of Sprint 16, you should understand:

- `Date`
- `new Date()`
- current dates
- specific dates
- `Date.now()`
- Unix epoch
- UTC
- milliseconds
- `getDate()`
- `getMonth()`
- `getFullYear()`
- `getHours()`
- `getMinutes()`
- `getSeconds()`
- `toLocaleDateString()`
- `toISOString()`
- `Intl.DateTimeFormat`
- `new Audio()`
- `HTMLAudioElement`
- `src`
- `play()`
- `pause()`
- `currentTime`
- `canPlayType()`
- media formats
- MIME types
- source elements
- codecs
- `HTMLMediaElement`
- audio elements
- video elements
- media streams
- camera/microphone permission basics
- basic media controls

Part A: Current Progress Before Sprint 16

1. What You Already Know

Sprint 16 builds on the browser work from Sprints 11 to 15.

Earlier Sprint	What You Learned	How It Helps in Sprint 16
Sprint 11	Browser APIs, DOM, <code>document</code> , <code>window</code> , and <code>navigator</code>	Sprint 16 uses more browser APIs, especially media APIs.
Sprint 12	Changing page content safely	Sprint 16 often updates text like due dates or media status messages.
Sprint 13	Events and event listeners	Media controls usually run after clicks, such as Play, Pause, and Restart.
Sprint 14	Accessibility state	Media buttons and status messages should be understandable to users.
Sprint 15	Form control and validation	Date input and user-controlled values need careful checking.

Simple summary:

Sprints 11–15 taught you how to control the page. Sprint 16 teaches you how to use time and media tools provided by the browser.

Part B: The Main Idea of Sprint 16

2. Browser APIs for Time and Media

JavaScript itself has core language features like variables, arrays, objects, functions, and loops.

But browser JavaScript also gives you APIs.

An API is a set of tools you can use.

In Sprint 16, the browser/runtime gives you tools like:

```
new Date();
Date.now();
new Audio("sound.mp3");
audio.play();
audio.pause();
audio.canPlayType("audio/mpeg");
```

Simple meaning:

JavaScript can ask the browser for the current time, format dates, load media files, play audio, pause audio, and check media support.

Part C: Dates and Time

3. What Is a `Date` Object?

A `Date` object represents a specific moment in time.

Example:

```
const now = new Date();  
console.log(now);
```

Simple meaning:

`new Date()` creates a date object for the current date and time.

Possible output:

```
Wed May 06 2026 10:30:00 GMT...
```

The exact output depends on your computer, browser, locale, and time zone.

Beginner rule:

A `Date` object is not just a string. It is an object with methods for reading and formatting date/time information.

4. Creating the Current Date

Use `new Date()` with no arguments.

```
const now = new Date();  
  
console.log(now);
```

This creates a `Date` object for the current moment.

Simple meaning:

Create a date for right now.

5. Creating a Specific Date from a String

You can pass a date string into `new Date()`.

```
const birthday = new Date("2026-05-20");  
console.log(birthday);
```

Simple meaning:

Create a date object from the date string `"2026-05-20"`.

Beginner warning:

Date strings can sometimes behave differently depending on format and time zone. For simple practice, use clear formats and inspect the result.

6. Creating a Specific Date with Numbers

You can also create a date using numbers.

```
const birthday = new Date(2026, 4, 20);  
console.log(birthday);
```

This means:

```
year: 2026  
month index: 4  
day: 20
```

Important:

In this form, JavaScript months are zero-based.

Month indexes:

Month Index	Month
0	January
1	February
2	March
3	April
4	May
5	June
6	July
7	August
8	September
9	October
10	November
11	December

So this:

```
new Date(2026, 4, 20);
```

means:

May 20, 2026

not April 20, 2026.

Beginner rule:

Year and day are normal numbers. Month is zero-based.

7. Why `getMonth()` Is Confusing

`getMonth()` returns a month index from `0` to `11`.

Example:

```
const date = new Date(2026, 4, 20);  
  
console.log(date.getMonth());
```

Output:

4

But 4 means May.

This is confusing because humans usually count months from 1:

```
1 = January  
2 = February  
3 = March  
4 = April  
5 = May
```

JavaScript counts months from 0 in this API:

```
0 = January  
1 = February  
2 = March  
3 = April  
4 = May
```

Common beginner mistake:

```
const date = new Date(2026, 5, 20);
```

A beginner may think this means May 20, but it actually means June 20.

Safer habit:

```
const may = 4;  
const date = new Date(2026, may, 20);
```

This makes your intention clearer.

8. Getting Parts of a Date

JavaScript provides methods to read parts of a date.

```
const now = new Date();

console.log(now.getFullYear());
console.log(now.getMonth());
console.log(now.getDate());
console.log(now.getHours());
console.log(now.getMinutes());
console.log(now.getSeconds());
```

Method	What It Returns	Beginner Meaning
<code>getFullYear()</code>	year	Gets the full year, like <code>2026</code> .
<code>getMonth()</code>	month index	Gets the month from <code>0</code> to <code>11</code> .
<code>getDate()</code>	day of month	Gets the day number inside the month.
<code>getHours()</code>	hour	Gets the hour.
<code>getMinutes()</code>	minutes	Gets the minutes.
<code>getSeconds()</code>	seconds	Gets the seconds.

Example:

```
const date = new Date(2026, 4, 20, 9, 30, 15);

console.log(date.getFullYear()); // 2026
console.log(date.getMonth());    // 4
console.log(date.getDate());      // 20
console.log(date.getHours());     // 9
console.log(date.getMinutes());   // 30
console.log(date.getSeconds());   // 15
```

Simple meaning:

These methods let you pull individual pieces out of a date object.

9. `getDate()` vs Date Object

This line creates the full date object:


```
const now = new Date();
```

This line gets only the day of the month:

```
console.log(now.getDate());
```

Example:

```
const date = new Date(2026, 4, 20);  
console.log(date.getDate());
```

Output:

```
20
```

Simple meaning:

`getDate()` does not return the whole date. It returns the day number inside the month.

10. `Date.now()`

`Date.now()` gives you a number.

```
const timestamp = Date.now();  
console.log(timestamp);
```

It returns the number of milliseconds since:

```
January 1, 1970, 00:00:00 UTC
```

This starting point is called the Unix epoch.

Simple meaning:

`Date.now()` tells you how many milliseconds have passed since a standard computer-time starting point.

Important:

- `new Date()` gives a `Date` object.
- `Date.now()` gives a number.

Example:

```
const nowObject = new Date();  
const nowNumber = Date.now();  
  
console.log(nowObject);  
console.log(nowNumber);
```

Beginner rule:

Use `new Date()` when you need a date object. Use `Date.now()` when you need a timestamp number.

11. Unix Epoch

The Unix epoch is the standard starting point used by many computer systems for measuring time.

The Unix epoch is:

January 1, 1970, 00:00:00 UTC

When JavaScript gives you a timestamp like this:

1778062200000

It means:

This many milliseconds have passed since the Unix epoch.

Simple meaning:

Unix epoch time is a big number that represents time as milliseconds since January 1, 1970 UTC.

12. Milliseconds

A millisecond is one thousandth of a second.

```
1000 milliseconds = 1 second
```

So if two timestamps are `3000` milliseconds apart, they are 3 seconds apart.

Example:

```
const start = Date.now();

// Some code runs here.

const end = Date.now();
const difference = end - start;

console.log(difference);
```

Simple meaning:

`end - start` tells you how many milliseconds passed.

13. Measuring How Long Code Takes

You can use `Date.now()` to measure time.

```
const start = Date.now();

for (let i = 0; i < 1000000; i++) {
  // repeat empty work
}

const end = Date.now();

console.log(`Time passed: ${end - start} ms`);
```

Simple meaning:

Save the start time, run code, save the end time, subtract.

14. UTC

UTC means Coordinated Universal Time.

Beginner meaning:

UTC is a standard global time reference.

Why it matters:

- Your local time depends on your time zone.
- UTC is used as a common reference point.
- `toISOString()` outputs a UTC-based date string.
- Unix epoch is measured from midnight UTC on January 1, 1970.

Simple example:

```
const now = new Date();  
console.log(now.toISOString());
```

Possible output:

```
2026-05-06T09:30:00.000Z
```

The `Z` means UTC time.

Beginner rule:

Local display may depend on the user's location. ISO strings are usually UTC-based.

15. Formatting Dates

Raw `Date` objects are not always user-friendly.

Raw output may look like:

```
Wed May 06 2026 10:30:00 GMT...
```

A user-friendly date may look like:

May 6, 2026

JavaScript gives you formatting tools.

16. `toLocaleDateString()`

`toLocaleDateString()` formats a date for the user's locale.

Example:

```
const now = new Date();  
console.log(now.toLocaleDateString());
```

Possible output:

5/6/2026

Simple meaning:

Convert a date object into a readable date string.

You can pass options:

```
const date = new Date(2026, 4, 20);  
  
console.log(date.toLocaleDateString("en-US", {  
  year: "numeric",  
  month: "long",  
  day: "numeric"  
}));
```

Possible output:

May 20, 2026

Beginner rule:

Use `toLocaleDateString()` when you want a readable date for display.

17. `toISOString()`

`toISOString()` converts a date to a standard ISO string.

Example:

```
const date = new Date(2026, 4, 20);  
console.log(date.toISOString());
```

Possible output:

```
2026-05-20T04:00:00.000Z
```

The exact time part can differ because the original date may be interpreted in local time and then converted to UTC.

Simple meaning:

`toISOString()` gives a standard machine-friendly date/time string.

Common use:

- storing dates
- sending dates to APIs
- comparing date/time values
- debugging exact timestamps

Beginner warning:

`toISOString()` is not mainly for pretty display. It is more machine-friendly than user-friendly.

18. `Intl.DateTimeFormat`

`Intl.DateTimeFormat` gives more control over date formatting.

Example:

```
const date = new Date(2026, 4, 20);  
  
const formatter = new Intl.DateTimeFormat("en-US", {  
  dateStyle: "full"
```

```
});  
  
console.log(formatter.format(date));
```

Possible output:

Wednesday, May 20, 2026

Simple meaning:

`Intl.DateTimeFormat` creates a reusable date formatter.

Another example:

```
const formatter = new Intl.DateTimeFormat("en-US", {  
  weekday: "long",  
  year: "numeric",  
  month: "long",  
  day: "numeric"  
});  
  
const date = new Date(2026, 4, 20);  
console.log(formatter.format(date));
```

Possible output:

Wednesday, May 20, 2026

Beginner rule:

Use `Intl.DateTimeFormat` when you need consistent formatting or reusable formatting logic.

19. Date Formatting Comparison

Tool	Returns	Good For
<code>new Date()</code>	Date object	Creating a date/time object.
<code>Date.now()</code>	number	Measuring time differences.
<code>toLocaleDateString()</code>	string	Displaying a readable date.

Tool	Returns	Good For
<code>toISOString()</code>	string	Standard machine-friendly date format.
<code>Intl.DateTimeFormat</code>	formatter object	Reusable and controlled date formatting.

20. Date Mini-Function: Format a Due Date

This is a practical Sprint 16 function.

```
function formatDueDate(year, monthIndex, day) {
  const dueDate = new Date(year, monthIndex, day);

  const formatter = new Intl.DateTimeFormat("en-US", {
    dateStyle: "medium"
  });

  return formatter.format(dueDate);
}

console.log(formatDueDate(2026, 4, 20));
```

Possible output:

May 20, 2026

Step-by-step:

```
function formatDueDate(year, monthIndex, day) {
```

Create a reusable function with three inputs.

```
const dueDate = new Date(year, monthIndex, day);
```

Create a date object.

```
const formatter = new Intl.DateTimeFormat("en-US", {
  dateStyle: "medium"
});
```


Create a formatter.

```
return formatter.format(dueDate);
```

Return the readable formatted date.

Beginner rule:

A function should return the formatted value so it can be reused.

Part D: Audio

21. What Is Audio in JavaScript?

JavaScript can control audio in the browser.

You can:

- create an audio object
- load an audio file
- play audio
- pause audio
- reset audio
- check if the browser can probably play a file type
- read or change the current playback time

Basic example:

```
const audio = new Audio("sound.mp3");
```

Simple meaning:

Create an audio player in JavaScript and load `sound.mp3`.

22. `new Audio()`

`new Audio()` creates an audio element object.

Example:

```
const audio = new Audio("sound.mp3");
```

This creates an `HTMLAudioElement`.

Simple meaning:

`audio` is now a JavaScript object that can control an audio file.

You can use methods and properties like:

```
audio.play();  
audio.pause();  
audio.currentTime;  
audio.src;
```

23. `HTMLAudioElement`

An `HTMLAudioElement` is an audio element object in the browser.

You can create it in JavaScript:

```
const audio = new Audio("sound.mp3");
```

Or you can select it from HTML:

```
<audio id="player" src="sound.mp3"></audio>
```

```
const player = document.getElementById("player");
```

Simple meaning:

Whether created with JavaScript or selected from HTML, the audio object gives you media controls.

24. `src`

`src` means source.

It stores the file path or URL for the media.

Example:

```
const audio = new Audio();  
audio.src = "sound.mp3";
```

This is similar to:

```
const audio = new Audio("sound.mp3");
```

Simple meaning:

`src` tells the audio element which file to load.

HTML example:

```
<audio src="sound.mp3" controls></audio>
```

25. `play()`

`play()` starts playback.

Example:

```
const audio = new Audio("sound.mp3");  
audio.play();
```

Simple meaning:

Start playing the audio.

Important browser rule:

Browsers often block audio that tries to play automatically before the user interacts with the page.

This may fail:

```
const audio = new Audio("sound.mp3");
audio.play();
```

because it tries to play immediately on page load.

Better beginner pattern:

```
const audio = new Audio("sound.mp3");
const playButton = document.getElementById("playButton");

playButton.addEventListener("click", () => {
  audio.play();
});
```

Simple meaning:

Wait for the user to click, then play the sound.

26. Why Browsers May Block Automatic Audio

Browsers try to protect users from annoying or unwanted autoplay audio.

Problem example:

```
const audio = new Audio("sound.mp3");
audio.play();
```

This runs as soon as the page loads.

The browser may block it because the user did not click or interact yet.

Better:

```
button.addEventListener("click", () => {
  audio.play();
});
```

Simple meaning:

User gesture first. Media playback second.

Beginner rule:

For practice projects, call `play()` inside a click event handler.

27. `pause()`

`pause()` pauses audio playback.

Example:

```
audio.pause();
```

Simple meaning:

Stop playing for now, but keep the current playback position.

If audio is at 10 seconds and you call:

```
audio.pause();
```

then later call:

```
audio.play();
```

it continues from around 10 seconds.

28. `currentTime`

`currentTime` stores the current playback position in seconds.

Example:

```
console.log(audio.currentTime);
```

If the audio is 12.5 seconds into playback, the output may be:

```
12.5
```

You can also change it:

```
audio.currentTime = 0;
```

Simple meaning:

Move playback to the beginning.

Another example:

```
audio.currentTime = 30;
```

Simple meaning:

Move playback to 30 seconds.

29. Resetting Audio

There is no normal beginner `stop()` method for audio.

To stop and reset audio, do this:

```
audio.pause();  
audio.currentTime = 0;
```

Simple meaning:

1. pause the audio
2. move the playback position back to the beginning

If you want to restart from the beginning immediately:

```
audio.pause();  
audio.currentTime = 0;  
audio.play();
```

Simple meaning:

1. pause
2. reset to the beginning
3. play again

Beginner rule:

`pause()` stops movement. `currentTime = 0` returns to the beginning.

30. Audio Button Example

HTML:

```
<button id="playButton">Play</button>
<button id="pauseButton">Pause</button>
<button id="restartButton">Restart</button>
```

JavaScript:

```
const audio = new Audio("sound.mp3");

const playButton = document.getElementById("playButton");
const pauseButton = document.getElementById("pauseButton");
const restartButton = document.getElementById("restartButton");

playButton.addEventListener("click", () => {
  audio.play();
});

pauseButton.addEventListener("click", () => {
  audio.pause();
});

restartButton.addEventListener("click", () => {
  audio.pause();
  audio.currentTime = 0;
  audio.play();
});
```

Step-by-step:

```
const audio = new Audio("sound.mp3");
```

Create the audio object.

```
const playButton = document.getElementById("playButton");
```

Find the play button.

```
playButton.addEventListener("click", () => {  
  audio.play();  
});
```

When the user clicks Play, play the audio.

```
pauseButton.addEventListener("click", () => {  
  audio.pause();  
});
```

When the user clicks Pause, pause the audio.

```
restartButton.addEventListener("click", () => {  
  audio.pause();  
  audio.currentTime = 0;  
  audio.play();  
});
```

When the user clicks Restart, reset and play the audio.

Part E: `canPlayType()`

31. Why `canPlayType()` Exists

Browsers do not support every media format equally.

A file may work in one browser but not another.

`canPlayType()` lets JavaScript ask the browser whether it can probably play a media type.

Example:


```
const audio = new Audio();  
  
console.log(audio.canPlayType("audio/mpeg"));
```

Simple meaning:

Browser, can you probably play MP3 audio?

32. `canPlayType()` Return Values

`canPlayType()` usually returns one of these:

Return Value	Meaning
<code>"probably"</code>	The browser is likely able to play it.
<code>"maybe"</code>	The browser might be able to play it.
<code>""</code>	The browser probably cannot play it.

Example:

```
const audio = new Audio();  
const result = audio.canPlayType("audio/mpeg");  
  
console.log(result);
```

Possible output:

probably

or:

maybe

or an empty string.

33. Using `canPlayType()` in a Condition

```
const audio = new Audio();
const result = audio.canPlayType("audio/mpeg");

if (result === "") {
  console.log("This browser may not support MP3.");
} else {
  console.log("This browser can probably play MP3.");
}
```

Simple meaning:

If the result is empty, the browser probably cannot play the type. Otherwise, it has some level of support.

Beginner rule:

Use `canPlayType()` to check support, not to play the file.

Part F: Media Formats, MIME Types, and Codecs

34. What Is a Media Format?

A media format is the kind of media file.

Examples:

```
.mp3
.mp4
.webm
.ogg
```

Simple meaning:

The format is the file type or container used for the media.

Common examples:

Extension	Common Use
<code>.mp3</code>	audio
<code>.ogg</code>	audio
<code>.mp4</code>	video/audio container
<code>.webm</code>	video/audio container

35. What Is a MIME Type?

A MIME type tells the browser what kind of file it is handling.

Examples:

```
text/html
application/json
audio/mpeg
audio/mp4
video/mp4
```

Simple meaning:

A MIME type is a label that tells the browser what kind of content a file contains.

Common MIME types:

File Type	MIME Type
HTML	<code>text/html</code>
JSON	<code>application/json</code>
MP3 audio	<code>audio/mpeg</code>
MP4 audio	<code>audio/mp4</code>
MP4 video	<code>video/mp4</code>
WebM video	<code>video/webm</code>
Ogg audio	<code>audio/ogg</code>

Example:

```
<source src="sound.mp3" type="audio/mpeg">
```

Simple meaning:

This source file is MP3 audio.

36. Why MIME Types Matter

The browser uses MIME types to decide how to handle a file.

For example:

```
<source src="video.mp4" type="video/mp4">
```

This tells the browser:

This file is video in MP4 format.

If the MIME type is wrong, the browser may fail to load or play the media correctly.

Beginner rule:

The file extension and MIME type should match the real content.

37. What Is a Codec?

A codec is used to encode and decode media data.

Simple meaning:

A codec is the method used to compress and read the audio or video inside a file.

Think of it this way:

```
format/container = the box  
codec = how the media inside the box is packed and unpacked
```

Example idea:

Two files can both be `.mp4`, but they may use different codecs inside.

Beginner rule:

You do not need to master codecs yet, but you should know that format and codec are not always the same thing.

Part G: Audio and Video Elements

38. The HTML `<audio>` Element

HTML can create an audio player.

Example:

```
<audio controls>
  <source src="sound.mp3" type="audio/mpeg">
  <source src="sound.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>
```

Simple meaning:

Create an audio player and let the browser choose a supported source.

Important parts:

Part	Meaning
<code><audio></code>	Creates an audio element.
<code>controls</code>	Shows built-in browser controls.
<code><source></code>	Gives the browser a media file option.
<code>src</code>	File path or URL.
<code>type</code>	MIME type.
fallback text	Text shown if audio is unsupported.

39. The HTML `<video>` Element

HTML can also create a video player.

Example:

```
<video controls width="400">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support video.
</video>
```

Simple meaning:

Create a video player and let the browser choose a supported source.

Important parts:

Part	Meaning
<code><video></code>	Creates a video element.
<code>controls</code>	Shows built-in video controls.
<code>width="400"</code>	Sets display width.
<code><source></code>	Gives media source options.
<code>src</code>	File path or URL.
<code>type</code>	MIME type.
fallback text	Text shown if video is unsupported.

40. Why Use Multiple `<source>` Elements?

Not all browsers support the same formats.

Example:

```
<audio controls>
  <source src="sound.mp3" type="audio/mpeg">
  <source src="sound.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>
```

Simple meaning:

Try MP3. If that is not supported, try Ogg.

The browser chooses the first source it can play.

Beginner rule:

Use multiple sources when you want better browser compatibility.

41. The `controls` Attribute

The `controls` attribute tells the browser to show built-in media controls.

Example:

```
<audio controls src="sound.mp3"></audio>
```

For video:

```
<video controls src="video.mp4"></video>
```

Built-in controls usually include:

- play
- pause
- progress bar
- time display
- volume
- fullscreen for video

Simple meaning:

`controls` gives the user a ready-made media player interface.

Part H: `HTMLMediaElement`

42. What Is `HTMLMediaElement`?

`HTMLMediaElement` is a shared browser interface for audio and video elements.

Simple meaning:

Audio and video elements share many of the same media controls.

Both of these are media elements:

```
<audio src="sound.mp3"></audio>
<video src="video.mp4"></video>
```

They both can use tools like:

```
play();
pause();
currentTime;
src;
controls;
muted;
volume;
```

43. Common Media Properties and Methods

Tool	Meaning
<code>play()</code>	Starts playback.
<code>pause()</code>	Pauses playback.
<code>currentTime</code>	Current playback position in seconds.
<code>src</code>	Media file URL or path.
<code>controls</code>	Whether browser controls are shown.
<code>muted</code>	Whether the media is muted.
<code>volume</code>	Volume from <code>0</code> to <code>1</code> .

Example:

```
player.volume = 0.5;
player.muted = true;
player.currentTime = 0;
```

Simple meaning:

- `volume = 0.5` means 50% volume.
- `muted = true` means no sound.
- `currentTime = 0` means return to the beginning.

44. Selecting an Existing Audio Element

HTML:

```
<audio id="player" src="sound.mp3"></audio>

<button id="play">Play</button>
<button id="pause">Pause</button>
```

JavaScript:

```
const player = document.getElementById("player");
const play = document.getElementById("play");
const pause = document.getElementById("pause");

play.addEventListener("click", () => {
  player.play();
});

pause.addEventListener("click", () => {
  player.pause();
});
```

Simple meaning:

The audio element already exists in HTML. JavaScript selects it and controls it.

45. `new Audio()` vs `<audio>` in HTML

Approach	Example	Good For
<code>new Audio()</code>	<pre>const audio = new Audio("sound.mp3")</pre>	Simple sound effects or audio controlled only by JavaScript.
<code><audio></code> element	<pre><audio id="player" src="sound.mp3"></audio></pre>	Audio visible in HTML or controlled by both HTML and JS.

Simple rule:

Use `<audio controls>` when you want the browser to show controls. Use `new Audio()` when JavaScript should create and manage the audio object.

Part I: Media Streams

46. What Is a Media Stream?

A media stream is live audio or video data.

Examples:

- microphone audio
- camera video
- screen recording stream
- video call stream

Simple meaning:

A media stream is not just a file. It is live media data coming from a device or source.

47. Common Media Stream Use Cases

Media streams are used for things like:

- video calls
- voice recorders
- camera previews
- screen recording
- microphone input
- webcam input

Beginner meaning:

Media streams let a web page work with live audio or video instead of only saved files.

48. Camera and Microphone Permission

Camera and microphone access is sensitive.

The browser does not normally allow a website to use the camera or microphone silently.

The user must give permission.

Simple rule:

Never assume camera or microphone access is automatic.

Conceptual flow:

```
JavaScript asks for camera/microphone access
↓
Browser asks the user for permission
↓
User allows or blocks
↓
JavaScript receives a stream or an error
```

49. Beginner Media Stream Example

A common media stream method is:

```
navigator.mediaDevices.getUserMedia()
```

Simple example:

```
async function startCamera() {
  const stream = await navigator.mediaDevices.getUserMedia({
    video: true,
    audio: false
  });

  const video = document.getElementById("preview");
  video.srcObject = stream;
}
```

HTML:

```
<video id="preview" autoplay playsinline></video>
<button id="startCamera">Start Camera</button>
```

Safer event-based setup:

```
const button = document.getElementById("startCamera");

button.addEventListener("click", () => {
```

```
startCamera();  
});
```

Simple meaning:

After the user clicks, ask for camera access. If allowed, show the camera stream in the video element.

Beginner note:

This is concept-level for Sprint 16 unless your project specifically asks for camera or microphone work.

Part J: Other Media APIs

50. Other Media APIs at a Basic Awareness Level

Sprint 16 mentions that there are other media APIs.

You do not need to master them yet.

Examples of media-related browser features include:

- audio playback
- video playback
- media streams
- camera capture
- microphone capture
- screen capture
- browser media controls

Beginner target:

Know that the browser has many media tools, but focus first on `Audio`, `<audio>`, `<video>`, `play()`, `pause()`, `currentTime`, formats, MIME types, and basic stream permission behavior.

Part K: Full Beginner Mini-Build

51. Mini-Build Goal

Build a small page with two features:

1. a due-date formatter
2. play, pause, and restart buttons for one audio file

You will practice:

- selecting elements
- using `Date`
- using `Intl.DateTimeFormat`
- using `new Audio()`
- using `play()`
- using `pause()`
- using `currentTime`
- updating page text with `textContent`
- using click events

52. Full HTML File

Create one HTML file.

Important:

This example expects a file named `sound.mp3` in the same folder as the HTML file.

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Sprint 16 Practice</h1>

    <section>
      <h2>Due Date Formatter</h2>

      <p id="dueDateOutput">Due date will appear here.</p>

      <button id="showDueDateButton">Show Due Date</button>
    </section>

    <section>
```

```

<h2>Audio Controls</h2>

<button id="playButton">Play</button>
<button id="pauseButton">Pause</button>
<button id="restartButton">Restart</button>

<p id="audioStatus">Audio not started.</p>
</section>

<script>
  const dueDateOutput = document.getElementById("dueDateOutput");
  const showDueDateButton = document.getElementById("showDueDateButton");

  function formatDueDate(year, monthIndex, day) {
    const dueDate = new Date(year, monthIndex, day);

    const formatter = new Intl.DateTimeFormat("en-US", {
      dateStyle: "full"
    });

    return formatter.format(dueDate);
  }

  showDueDateButton.addEventListener("click", () => {
    const formattedDate = formatDueDate(2026, 4, 20);
    dueDateOutput.textContent = `Task is due on ${formattedDate}.`;
  });

  const audio = new Audio("sound.mp3");

  const playButton = document.getElementById("playButton");
  const pauseButton = document.getElementById("pauseButton");
  const restartButton = document.getElementById("restartButton");
  const audioStatus = document.getElementById("audioStatus");

  playButton.addEventListener("click", () => {
    audio.play();
    audioStatus.textContent = "Audio playing.";
  });

  pauseButton.addEventListener("click", () => {
    audio.pause();
    audioStatus.textContent = "Audio paused.";
  });

  restartButton.addEventListener("click", () => {
    audio.pause();
    audio.currentTime = 0;
  });

```

```
        audio.play();
        audioStatus.textContent = "Audio restarted.";
    });
</script>
</body>
</html>
```

53. Mini-Build Explanation

Due Date Section

```
const dueDateOutput = document.getElementById("dueDateOutput");
const showDueDateButton = document.getElementById("showDueDateButton");
```

These lines select the paragraph and button.

```
function formatDueDate(year, monthIndex, day) {
```

This creates a reusable function.

```
const dueDate = new Date(year, monthIndex, day);
```

This creates a date object.

```
const formatter = new Intl.DateTimeFormat("en-US", {
    dateStyle: "full"
});
```

This creates a formatter.

```
return formatter.format(dueDate);
```

This returns the formatted date.

```
showDueDateButton.addEventListener("click", () => {
```

This waits for the user to click the button.

```
dueDateOutput.textContent = `Task is due on ${formattedDate}.`;
```

This safely updates the paragraph text.

Audio Section

```
const audio = new Audio("sound.mp3");
```

This creates the audio object.

```
playButton.addEventListener("click", () => {  
  audio.play();  
});
```

This plays audio after a click.

```
pauseButton.addEventListener("click", () => {  
  audio.pause();  
});
```

This pauses audio.

```
restartButton.addEventListener("click", () => {  
  audio.pause();  
  audio.currentTime = 0;  
  audio.play();  
});
```

This restarts audio from the beginning.

Part L: Warm-Up Drills

54. Date Drills

Drill 1: Create the Current Date

Task:

Create a `Date` object for now and print it.

Answer:

```
const now = new Date();  
console.log(now);
```

Drill 2: Print the Current Year

Task:

Print the current year.

Answer:

```
const now = new Date();  
console.log(now.getFullYear());
```

Drill 3: Print Month Index

Task:

Create a date for May 20, 2026 and print its month index.

Answer:

```
const date = new Date(2026, 4, 20);  
console.log(date.getMonth());
```

Expected output:

```
4
```

Explanation:

May is month index `4` because month indexes start at `0`.

Drill 4: Format a Date

Task:

Format a date using `toLocaleDateString()`.

Answer:

```
const date = new Date(2026, 4, 20);
console.log(date.toLocaleDateString());
```

Drill 5: Use `Intl.DateTimeFormat`

Task:

Format a date using `Intl.DateTimeFormat`.

Answer:

```
const date = new Date(2026, 4, 20);

const formatter = new Intl.DateTimeFormat("en-US", {
  dateStyle: "full"
});

console.log(formatter.format(date));
```

Drill 6: Use `Date.now()`

Task:

Print the current Unix epoch timestamp in milliseconds.

Answer:

```
console.log(Date.now());
```

55. Audio Drills

Drill 1: Create an Audio Object

Task:

Create an audio object for `sound.mp3`.

Answer:

```
const audio = new Audio("sound.mp3");
```

Drill 2: Check MP3 Support

Task:

Use `canPlayType()` to check `audio/mpeg`.

Answer:

```
const audio = new Audio();  
console.log(audio.canPlayType("audio/mpeg"));
```

Drill 3: Play After Button Click

Task:

Play audio only after the user clicks a button.

Answer:

```
<button id="playButton">Play</button>
```

```
const audio = new Audio("sound.mp3");  
const playButton = document.getElementById("playButton");  
  
playButton.addEventListener("click", () => {
```

```
    audio.play();
  });
```

Drill 4: Pause Audio

Task:

Pause audio after a button click.

Answer:

```
<button id="pauseButton">Pause</button>
```

```
const pauseButton = document.getElementById("pauseButton");

pauseButton.addEventListener("click", () => {
  audio.pause();
});
```

Drill 5: Reset Audio

Task:

Reset audio to the beginning.

Answer:

```
audio.pause();
audio.currentTime = 0;
```

Part M: Debug Challenges

56. Debug Challenge 1: Month Is Wrong

Problem:

```
const date = new Date(2026, 5, 20);
console.log(date.toLocaleDateString());
```

A beginner expects May 20, 2026.

Actual issue:

Month index `5` means June, not May.

Correct version:

```
const date = new Date(2026, 4, 20);
console.log(date.toLocaleDateString());
```

Reason:

JavaScript month indexes start at `0`.

57. Debug Challenge 2: Audio Does Not Play on Page Load

Problem:

```
const audio = new Audio("sound.mp3");
audio.play();
```

Possible issue:

The browser blocks automatic playback because the user has not interacted with the page.

Better version:

```
<button id="playButton">Play</button>
```

```
const audio = new Audio("sound.mp3");
const playButton = document.getElementById("playButton");

playButton.addEventListener("click", () => {
  audio.play();
});
```

Reason:

Many browsers require a user gesture before playing audio.

58. Debug Challenge 3: Restart Does Not Go Back to Beginning

Problem:

```
restartButton.addEventListener("click", () => {  
  audio.play();  
});
```

Issue:

This only plays the audio. It does not reset the playback position.

Correct version:

```
restartButton.addEventListener("click", () => {  
  audio.pause();  
  audio.currentTime = 0;  
  audio.play();  
});
```

Reason:

`currentTime = 0` moves playback back to the beginning.

59. Debug Challenge 4: Wrong MIME Type

Problem:

```
<source src="sound.mp3" type="video/mp4">
```

Issue:

The file is MP3 audio, but the MIME type says MP4 video.

Better:

```
<source src="sound.mp3" type="audio/mpeg">
```

Reason:

MIME types should describe the real content type.

Part N: Common Beginner Mistakes

60. Date Mistakes

Common mistakes:

1. Thinking `getMonth()` returns `1` to `12`.
2. Forgetting that month indexes start at `0`.
3. Using `Date.now()` and expecting a readable date string.
4. Using raw `Date` output for user-facing display.
5. Forgetting that `toISOString()` uses UTC.
6. Treating a `Date` object like a plain string.

Better habits:

- Remember: January is `0`.
 - Use `toLocaleDateString()` for simple user display.
 - Use `Intl.DateTimeFormat` for controlled formatting.
 - Use `Date.now()` for timestamps and time differences.
 - Use `toISOString()` for standard machine-friendly output.
-

61. Audio and Video Mistakes

Common mistakes:

1. Calling `audio.play()` automatically on page load.
2. Forgetting that browsers may block autoplay.
3. Expecting `pause()` to reset audio.
4. Forgetting to set `currentTime = 0` when restarting.
5. Using the wrong MIME type.
6. Assuming all browsers support all formats.
7. Confusing format and codec.
8. Assuming camera/microphone access is automatic.

Better habits:

- Call `play()` after a click.
- Use `pause()` to pause.
- Use `currentTime = 0` to reset.
- Use `<source>` elements for multiple formats.
- Use correct MIME types.
- Remember that media streams require permission.

Part O: Concept Summary

62. Date Concept Summary

Concept	Simple Meaning
<code>Date</code>	Object representing a moment in time.
<code>new Date()</code>	Creates a date for now.
<code>new Date("2026-05-20")</code>	Creates a date from a string.
<code>new Date(2026, 4, 20)</code>	Creates May 20, 2026.
<code>getFullYear()</code>	Gets the year.
<code>getMonth()</code>	Gets month index from <code>0</code> to <code>11</code> .
<code>getDate()</code>	Gets the day of the month.
<code>getHours()</code>	Gets the hour.
<code>getMinutes()</code>	Gets the minutes.
<code>getSeconds()</code>	Gets the seconds.
<code>Date.now()</code>	Gets current timestamp in milliseconds.
Unix epoch	January 1, 1970, 00:00:00 UTC.
UTC	Global time reference.
<code>toLocaleDateString()</code>	Formats a date for display.
<code>toISOString()</code>	Creates a standard UTC ISO string.
<code>Intl.DateTimeFormat</code>	Creates a reusable formatter.

63. Media Concept Summary

Concept	Simple Meaning
<code>new Audio()</code>	Creates an audio element object.
<code>HTMLAudioElement</code>	Browser audio element object.
<code>src</code>	Media file path or URL.
<code>play()</code>	Starts media playback.
<code>pause()</code>	Pauses media playback.
<code>currentTime</code>	Current playback position in seconds.
<code>canPlayType()</code>	Checks whether browser may support a media type.
Format	File/container type like <code>.mp3</code> , <code>.mp4</code> , <code>.webm</code> .
MIME type	Browser content label like <code>audio/mpeg</code> .
Codec	Method used to encode/decode media.
<code><source></code>	Gives browser media file options.
<code><audio></code>	HTML audio player element.
<code><video></code>	HTML video player element.
<code>HTMLMediaElement</code>	Shared audio/video media interface.
Media stream	Live audio/video data.
Permission	User approval needed for camera/microphone.

Part P: Minimum Passing Standard

64. You Are Ready for Sprint 17 When You Can Explain These

You should be able to answer these without looking back:

1. What does `new Date()` create?
2. What does `Date.now()` return?
3. What is the Unix epoch?
4. What does UTC mean in beginner terms?
5. Why is `getMonth()` confusing?
6. What does `getFullYear()` return?
7. What does `getDate()` return?
8. What does `toLocaleDateString()` do?

9. What does `toISOString()` do?
 10. What does `Intl.DateTimeFormat` help with?
 11. What does `new Audio("sound.mp3")` create?
 12. What is an `HTMLAudioElement`?
 13. What does `src` store?
 14. What does `play()` do?
 15. What does `pause()` do?
 16. What does `currentTime` represent?
 17. How do you reset audio to the beginning?
 18. Why might browsers block `audio.play()` on page load?
 19. What does `canPlayType()` check?
 20. What is a media format?
 21. What is a MIME type?
 22. What is a codec?
 23. Why use multiple `<source>` elements?
 24. What does the `controls` attribute do?
 25. What is `HTMLMediaElement`?
 26. What is a media stream?
 27. Why does camera/microphone access require permission?
 28. How do you build a due-date formatter?
 29. How do you add play, pause, and restart buttons for audio?
-

Part Q: Answer Key

65. Short Answers

1. `new Date()` creates a date object for the current date and time.
2. `Date.now()` returns the current timestamp in milliseconds.
3. The Unix epoch is January 1, 1970, 00:00:00 UTC.
4. UTC is a standard global time reference.
5. `getMonth()` is confusing because it returns `0` to `11`, not `1` to `12`.
6. `getFullYear()` returns the full year, like `2026`.
7. `getDate()` returns the day of the month.
8. `toLocaleDateString()` returns a readable date string for a locale.
9. `toISOString()` returns a standard UTC-based ISO date/time string.
10. `Intl.DateTimeFormat` helps create reusable and controlled date formatting.
11. `new Audio("sound.mp3")` creates an audio element object for that file.
12. An `HTMLAudioElement` is a browser audio element object with media controls.
13. `src` stores the media file path or URL.
14. `play()` starts playback.
15. `pause()` pauses playback.
16. `currentTime` is the current playback position in seconds.
17. Reset audio with `audio.pause(); audio.currentTime = 0;`.
18. Browsers may block autoplay because the user has not interacted with the page.

19. `canPlayType()` checks whether the browser can probably play a media type.
 20. A media format is the file/container type, like `.mp3` or `.mp4`.
 21. A MIME type tells the browser what kind of content the file contains.
 22. A codec encodes and decodes media data.
 23. Multiple `<source>` elements give the browser fallback media options.
 24. `controls` shows built-in browser media controls.
 25. `HTMLMediaElement` is the shared audio/video media interface.
 26. A media stream is live audio or video data.
 27. Camera/microphone access requires permission because it is sensitive device access.
 28. Build a due-date formatter with `new Date()` plus a formatting method.
 29. Add buttons, select them, then call `play()`, `pause()`, and `currentTime = 0` inside event listeners.
-

Part R: Final Sprint 16 Summary

Sprint 16 teaches two main browser API areas:

1. date/time APIs
2. media APIs

The most important date ideas:

- `new Date()` creates a date object.
- `Date.now()` gives milliseconds since the Unix epoch.
- Unix epoch starts at January 1, 1970, 00:00:00 UTC.
- `getMonth()` returns `0` to `11`.
- `toLocaleDateString()` formats for humans.
- `toISOString()` formats for machines.
- `Intl.DateTimeFormat` gives controlled reusable formatting.

The most important media ideas:

- `new Audio()` creates an audio element object.
- `play()` starts playback.
- `pause()` pauses playback.
- `currentTime` controls the playback position.
- `pause()` plus `currentTime = 0` resets audio.
- Browsers may block autoplay until the user interacts.
- `canPlayType()` checks possible media support.
- MIME types tell the browser what kind of media file it is.
- Codecs are used to encode and decode media.
- `<source>` elements provide fallback media choices.
- `<audio>` and `<video>` share many `HTMLMediaElement` controls.
- Media streams handle live camera/microphone data and require permission.

Final beginner memory:

```
Date = work with time.  
Date.now() = milliseconds since Unix epoch.  
getMonth() = zero-based month index.  
Audio/Video = control media.  
MIME type = tell browser what kind of file it is.  
Codec = how media is encoded and decoded.  
currentTime = current playback position.  
pause() + currentTime = 0 = reset.  
play() should usually happen after a user click.  
Media streams need user permission.
```

Sprint 16 is not mainly about complex algorithms. It is about using built-in browser/runtime objects correctly. The main beginner mistake is treating dates and media like plain strings or numbers. They are objects with their own methods, browser rules, and limitations.